day 26 | 251029 W

Today we will review coupled first order differential equations. There are some significant changes we need to make to portions of our code, but at the end of the day these changes will set us up nicely for the next topic (second order differential equations!). The most important different is that we will be plugging in an array in the place of our dependent variables (like x and y), and this change will allow our program to handle the variation of these variables all at once, a rather elegant solution but one that requires you to really follow what is going on. In class, you will likely have time to complete the homework assignment which is exercises 5.2 and 5.3.

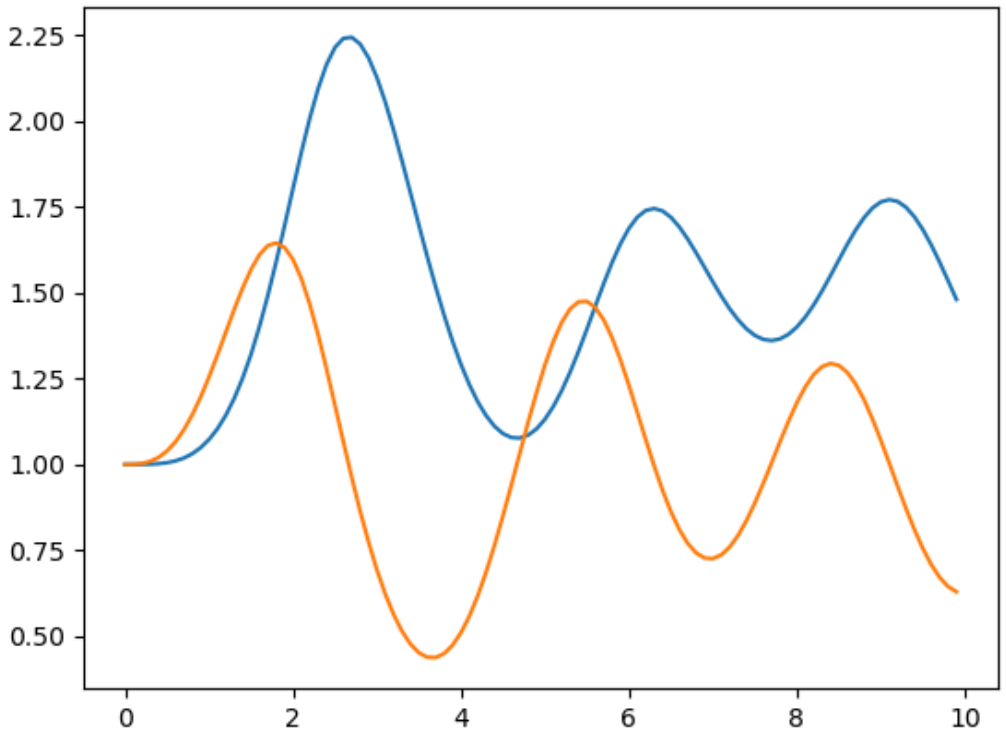
In [1]:
```
%matplotlib ipympl

import numpy as np
import matplotlib.pyplot as plt
```

In [5]:
```
# define the differential equation
def f(r, t): # don't forget r is a vector!
    x = r[0]
    y = r[1]
    w = 1.0
    fx = x*y-x  # this is the dx/dt equation!
    fy = y - x*y + np.sin(w*t)**2  # this is the dy/dt equation!
    return(np.array([fx, fy]))

# define the boundary condition
a = 0.0
b = 10.0
N = 100
dt = (b-a)/N

# Runge-Kutta 4th order

r = np.array([1.0, 1.0]) # initial condition

tpoints = np.arange(a, b, dt)

xpoints = []
ypoints = []

for i in tpoints:
    xpoints.append(r[0])
    ypoints.append(r[1])
    k1 = dt*f(r, i)
    k2 = dt*f(r+1/2*k1, i+1/2*dt)
    k3 = dt*f(r+1/2*k2, i+1/2*dt)
    k4 = dt*f(r+k3, i+dt)
    r = r + (k1+2*k2+2*k3+k4)/6
```

```
fig0, ax0 = plt.subplots()
ax0.plot(tpoints, xpoints)
ax0.plot(tpoints, ypoints)
```

Out[5]: [<matplotlib.lines.Line2D at 0x7667bf2964b0>]

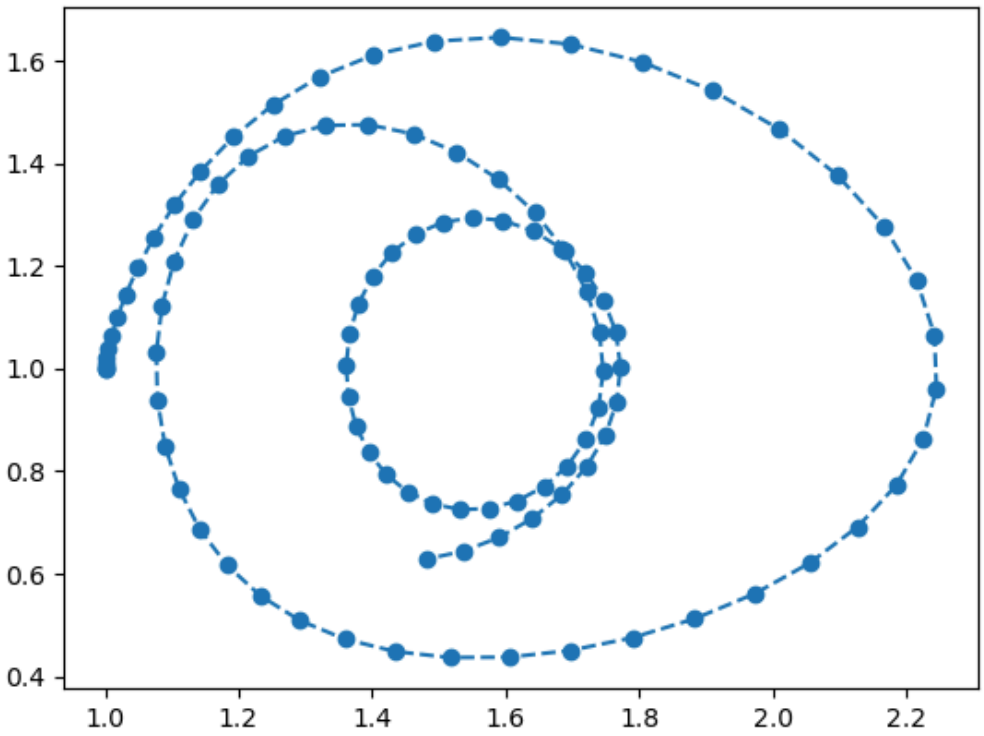
Figure



In [7]:
```
fig1, ax1 = plt.subplots()
ax1.plot(xpoints, ypoints, 'o--')
```

Out[7]: [<matplotlib.lines.Line2D at 0x7667bc9e77a0>]

Figure



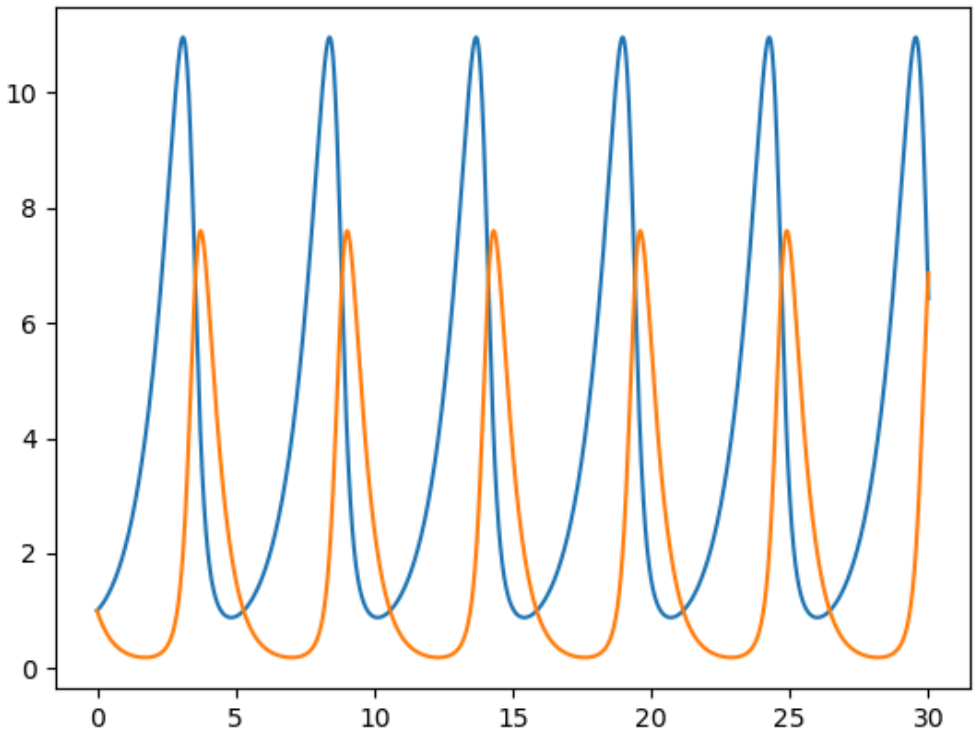
In [9]:

```
# define the differential equation
def f(r, t): # don't forget r is a vector!
    x = r[0]
    y = r[1]
    a = 1
    b = 0.5
    g = 0.5
    d = 2
    fx = a*x-b*x*y  # this is the dx/dt equation!
    fy = g*x*y-d*y  # this is the dy/dt equation!
    return(np.array([fx, fy]))

# define the boundary condition
a = 0.0
b = 30.0
N = 10000
dt = (b-a)/N

# Runge-Kutta 4th order

r = np.array([1.0, 1.0]) # initial condition

tpoints = np.arange(a, b, dt)

xpoints = []
ypoints = []
```

```
for i in tpoints:
    xpoints.append(r[0])
    ypoints.append(r[1])
    k1 = dt*f(r, i)
    k2 = dt*f(r+1/2*k1, i+1/2*dt)
    k3 = dt*f(r+1/2*k2, i+1/2*dt)
    k4 = dt*f(r+k3, i+dt)
    r = r + (k1+2*k2+2*k3+k4)/6

fig2, ax2 = plt.subplots()
ax2.plot(tpoints, xpoints)
ax2.plot(tpoints, ypoints)
```

Out[9]: [<matplotlib.lines.Line2D at 0x7667bf3710d0>]

Figure



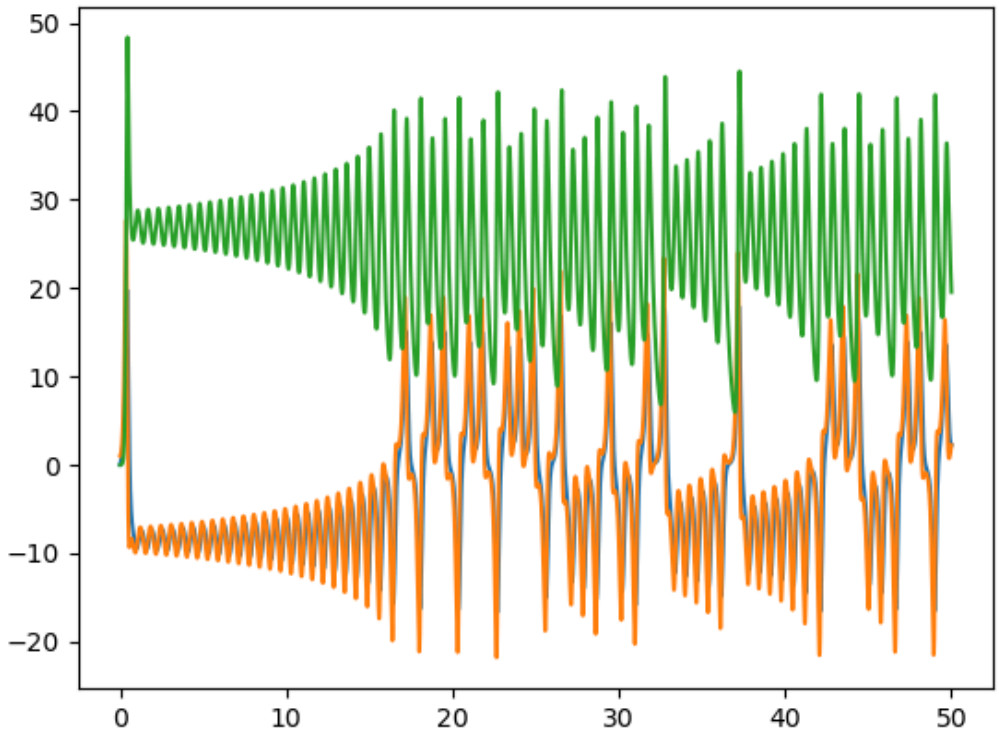
In [ ]:

In [14]:
```
# define the differential equation
def f(r, t): # don't forget r is a vector!
    x = r[0]
    y = r[1]
    z = r[2]
    sigma = 10
    rho = 28
    beta = 8/3.
    fx = sigma*(y-x) # this is the dx/dt equation!
    fy = rho*x - y - x*z  # this is the dy/dt equation!
```

```
    fz = x*y - beta*z # this is the dz/dt equation
    return(np.array([fx, fy, fz]))

# define the boundary condition
a = 0.0
b = 50.0
N = 10000
dt = (b-a)/N

# Runge-Kutta 4th order

r = np.array([0.0, 1.0, 0.0]) # initial condition

tpoints = np.arange(a, b, dt)

xpoints = []
ypoints = []
zpoints = []

for i in tpoints:
    xpoints.append(r[0])
    ypoints.append(r[1])
    zpoints.append(r[2])
    k1 = dt*f(r, i)
    k2 = dt*f(r+1/2*k1, i+1/2*dt)
    k3 = dt*f(r+1/2*k2, i+1/2*dt)
    k4 = dt*f(r+k3, i+dt)
    r = r + (k1+2*k2+2*k3+k4)/6

fig4, ax4 = plt.subplots()
ax4.plot(tpoints, xpoints)
ax4.plot(tpoints, ypoints)
ax4.plot(tpoints, zpoints)
```

Out[14]: `[<matplotlib.lines.Line2D at 0x7667bc5b25a0>]`

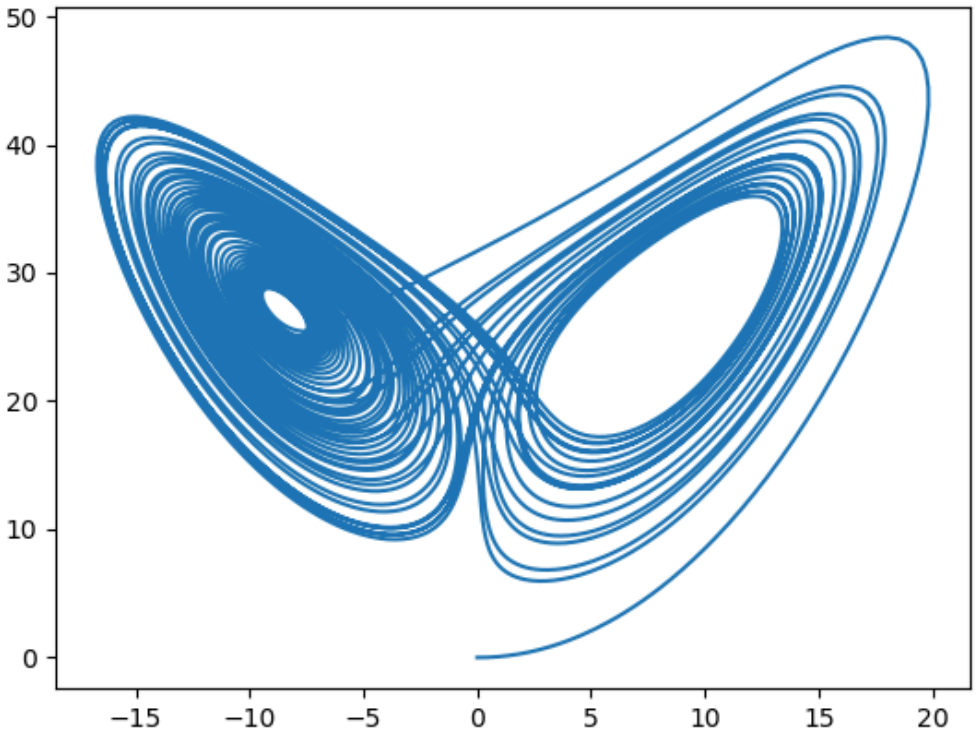
Figure



In [15]:
```
fig5, ax5 = plt.subplots(projection=)
ax5.plot(xpoints, zpoints)
```

Out[15]: [<matplotlib.lines.Line2D at 0x7667bc451c70>]

Figure



In [ ]: